

ECD-AIG Planning

src/ecd_aig 파이썬 소스 코드 해설서

응답 전 ECD-AIG 설계 추적과 위험 점검 코드를 읽기 위한 개발자 안내서

핵심 원칙

사전 점검 통과 != 경험적 타당도 확보

Generated for the local research prototype | 2026-06-01

파일 빠른 찾기

처음 읽을 때는 models.py → validation.py → item_quality.py → pre_response.py → __main__.py 순서를 권장한다.

범주	파일
핵심 흐름	models.py, validation.py, item_quality.py, pre_response.py, __main__.py
문항 준비	generation.py, import_items.py, blueprint.py, simulation.py
설명 산출물	ecd_report.py, caf.py, toulmin.py, dossier.py
응답자료 이후	scoring.py, response_data.py, psychometrics.py, agreement.py
출력과 화면	rendering.py, export.py, webapp.py, __init__.py

1. 이 문서의 목적

이 문서는 `src/ecd\_aig` 폴더의 파이썬 파일을 처음 읽는 사람을 위한 코드 해설서다.

현재 프로그램의 핵심 목적은 자동 생성 문항의 경험적 타당도를 확정하는 것이 아니다. 응답자료 수집 전에 문항의 설계 계보를 추적하고, 자동 규칙으로 탐지 가능한 위험을 찾고, 전문가 검토와 파일럿 조사를 준비하는 것이다.

자동 생성 또는 외부 반입 문항
 → ECD 계보 연결
 → 구조 점검
 → 문항 품질 점검
 → 전문가 검토
 → 파일럿 조사 준비

사전 점검 통과 != 경험적 타당도 확보

2. 전체 실행 흐름

사용자가 CLI 명령을 실행하면 `\_\_main\_\_.py`가 프로젝트 JSON을 읽고 기능별 모듈을 호출한다.

examples/*.json
 → models.load_project()
 → Project 객체
 → validation / item_quality / pre_response
 → blueprint / caf / toulmin / dossier
 → rendering 또는 export

응답자료가 생긴 이후에는 별도 흐름을 사용한다.

pilot_responses.csv
 → response_data.validate_responses()
 → scoring.score_response()
 → psychometrics.psychometrics_report()

이 두 번째 흐름은 후속 분석을 위한 기초 기능이다. 현재 코드만으로 완전한 IRT 또는 DIF 분석을 수행하지는 않는다.

3. 먼저 읽을 파일

처음 코드를 읽는다면 다음 순서가 좋다.

1. `models.py`: 프로젝트 데이터가 무엇인지 이해한다.
2. `validation.py`: 어떤 연결을 사전 점검하는지 본다.
3. `item\_quality.py`: 문항 문구의 위험을 어떻게 찾는지 본다.
4. `pre\_response.py`: 여러 점검 결과를 어떻게 하나의 readiness 상태로 합치는지 본다.

5. `\_\_main\_\_.py`: CLI 명령과 모듈 연결을 확인한다.

6. 나머지 보고서 및 후속 분석 모듈을 읽는다.

4. 처음 실행하는 순서

4.1 PowerShell 열기

Windows에서 PowerShell을 연다. 모든 명령은 프로젝트 폴더 `C:\dev\wecd`에서 실행한다.

4.2 프로젝트 폴더로 이동

```
cd C:\dev\wecd
```

4.3 파이썬 모듈 경로 설정

현재 PowerShell 창에서 한 번 실행한다. PowerShell 창을 닫았다가 다시 열면 다시 설정해야 한다.

```
$env:PYTHONPATH='src'
```

4.4 설치 상태 확인

```
py --version
```

`Python 3.x.x` 형태의 버전이 나오면 다음 단계로 진행한다.

4.5 자동 테스트 실행

```
py -m unittest discover -s tests -v
```

정상 상태에서는 마지막에 `OK`가 출력된다.

4.6 응답 전 통합 사전점검 실행

가장 먼저 사용할 기본 명령이다.

```
py -m ecd_aig pre-response examples\job_stress_workload_12item_user_project.json --markdown
```

주요 상태값:

```
revision_required
ready_for_expert_review
ready_for_pilot_administration
```

`ready\_for\_pilot\_administration`도 경험적 타당도 확보를 뜻하지 않는다.

4.7 웹 화면 실행

```
py -m ecd_aig webapp
```

PowerShell 창을 닫지 않은 상태에서 브라우저로 아래 주소를 연다.

```
http://127.0.0.1:8765
```

웹 화면의 `Example project` 메뉴에서 `examples` 폴더의 프로젝트를 선택할 수 있다.

서버를 종료하려면 PowerShell 창에서 `Ctrl+C`를 누른다.

4.8 세부 사전점검 명령

구조 계보 검사:

```
py -m ecd_aig validate examples\job_stress_workload_12item_user_project.json --markdown
```

문항 품질 자동 스크리닝:

```
py -m ecd_aig audit-items examples\job_stress_workload_12item_user_project.json --strict --markdown
```

blueprint 커버리지:

```
py -m ecd_aig blueprint examples\job_stress_workload_12item_user_project.json --markdown
```

CAF 설계 지도:

```
py -m ecd_aig caf examples\job_stress_workload_12item_user_project.json --markdown
```

특정 문항 dossier:

```
py -m ecd_aig dossier examples\job_stress_workload_12item_user_project.json --item JS-WB-012 --markdown
```

특정 문항 Toulmin 논증:

```
py -m ecd_aig toulmin examples\job_stress_workload_12item_user_project.json --item JS-WB-012 --markdown
```

4.9 부모 템플릿에서 후보 문항 생성

샘플 프로젝트에서 후보 문항 3개를 생성해 화면에 출력한다.

```
py -m ecd_aig generate examples\sample_project.json --template tpl_worry_001 --count 3 --markdown
```

프로젝트 JSON 파일에 생성 결과를 실제로 추가하려면 `--write`를 붙인다. 원본 파일이 변경되므로 먼저 복사본에서 실행하는 것을 권장한다.

```
Copy-Item examples\sample_project.json outputs\sample_project_working.json
```

```
py -m ecd_aig generate outputs\sample_project_working.json --template tpl_worry_001 --count 3 --write --markdown
```

4.10 후보 문항 내보내기

CSV:

```
py -m ecd_aig export-items examples\job_stress_workload_12item_user_project.json --out outputs\items.csv --format csv --json
```

JSON:

```
py -m ecd_aig export-items examples\job_stress_workload_12item_user_project.json --out outputs\items.json --format json --json
```

QTI-lite XML:

```
py -m ecd_aig export-items examples\job_stress_workload_12item_user_project.json --out outputs\items-qti-lite.xml --format qti-lite --json
```

`qti-lite`는 정식 IMS QTI 운영 패키지가 아니라 프로토타입 교환 형식이다.

4.11 응답자료가 들어온 이후에만 실행

응답 CSV 형식과 점수화 확인:

```
py -m ecd_aig responses examples\job_stress_workload_12item_user_project.json examples\pilot_responses.csv --json
```

기초 CTT 분석:

```
py -m ecd_aig psychometrics examples\job_stress_workload_12item_user_project.json examples\pilot_responses.csv --markdown
```

결측 응답 경고 예시:

```
py -m ecd_aig psychometrics examples\job_stress_workload_12item_user_project.json examples\pilot_responses_high_missing.csv --markdown
```

응답자료 이전 단계에서는 신뢰도, 문항-총점 상관, IRT, DIF 결과를 주장하지 않는다.

4.12 최소 실행 흐름 요약

처음에는 아래 네 단계만 실행하면 된다.

```
cd C:\dev\ecd
```

```
$env:PYTHONPATH='src'
```

```
py -m unittest discover -s tests -v
```

```
py -m ecd_aig pre-response examples\job_stress_workload_12item_user_project.json --markdown
```

웹 화면이 필요하다면 이어서 실행한다.

```
py -m ecd_aig webapp
```

5. 핵심 데이터 구조

`models.Project`가 프로그램 전체의 중심이다.

```
Project
├── constructs
├── ksas
├── evidence_claims
├── task_models
├── parent_templates
├── items
├── response_scale
├── reviews
└── metadata
```

문항 하나는 최소한 다음 계보를 가져야 한다.

```
item.construct_id
item.ksa_id
item.evidence_claim_id
item.task_model_id
item.parent_template_id
```

6. 파일별 코드 설명

`\_\_init\_\_.py`

역할: 파이썬이 `ecd\_aig` 폴더를 패키지로 인식하게 한다.

현재는 패키지 설명 문자열과 `\_\_version\_\_ = "0.1.0"`만 포함한다. 실행 로직은 없다.

`\_\_main\_\_.py`

역할: CLI 진입점이다. `py -m ecd\_aig ...` 명령을 해석한다.

핵심 흐름:

```
argparse로 하위 명령 파싱
→ 프로젝트 파일 로딩
→ 명령별 함수 호출
→ JSON 또는 Markdown 출력
```

주요 함수:

- `main()`: 모든 CLI 명령을 등록하고 분기한다.
- `print\_report()`: JSON 또는 Markdown 출력 방식을 통일한다.
- `validation\_markdown()`: 구조 게이트 결과를 표로 만든다.
- `generated\_markdown()`: 생성 문항 목록을 표로 만든다.

읽을 때 볼 부분: `pre-response`, `validate`, `audit-items`, `dossier`는 응답 전 흐름이고, `responses`, `psychometrics`는 응답자료 이후 흐름이다.

`models.py`

역할: 프로젝트 JSON을 파이썬 객체로 읽고 다시 저장한다.

주요 구성:

- `Project`: 데이터 클래스. ECD 설계와 문항을 한 객체에 담는다.
- `Project.from\_dict()`: JSON 딕셔너리를 `Project`로 바꾼다.
- `Project.to\_dict()`: `Project`를 저장 가능한 딕셔너리로 바꾼다.
- `ids()`: 특정 컬렉션의 ID 집합을 만든다.
- `item\_by\_id()`: 문항 ID로 문항을 찾는다.

- `template_by_id()`: 템플릿 ID로 부모 템플릿을 찾는다.
- `load_project()`, `save_project()`: UTF-8 JSON 파일 입출력 함수다.
- `result()`: 게이트 결과의 공통 형태를 만든다.

주의점: 'Project'는 유연한 딕셔너리 목록을 사용한다. 프로토타입에는 편하지만, 필드 오타를 생성 시점에 강하게 막지는 못한다.

`generation.py`

역할: 부모 템플릿과 변수 도메인으로 후보 문항을 생성한다.

주요 함수:

- `generate_items()`: `itertools.product()`로 변수 조합을 만들고 문항 stem에 값을 채운다.
- `generate_with_validation()`: 새 문항을 기존 프로젝트에 임시로 붙여 구조 검증을 실행한다.

예시:

```
stem_template = "나는 {work_situation} {negative_response}을 느낀다."
work_situation = ["마감이 가까워질 때"]
negative_response = ["압박감", "긴장감"]
→ 후보 문항 2개
```

주의점: 여기서 생성된 문항은 검증 완료 문항이 아니다. 구조 점검을 거친 후보 문항이다.

`import_items.py`

역할: 외부에서 만든 후보 문항 JSON을 프로젝트 형식으로 가져온다.

주요 함수:

- `import_candidate_items()`: 외부 'items'를 읽어 'Project'로 감싼다.
- `default_response_scale()`: 척도가 없을 때 사용할 기본 5점 Likert 척도를 제공한다.

중요한 설계 원칙: 외부 문항의 'construct_id', 'ksa_id', 'evidence_claim_id'를 코드가 추측하지 않는다. 값이 없으면 누락 상태로 유지한다. 이후 'validation.py'가 누락을 보고한다.

`validation.py`

역할: 응답 전 구조 게이트를 실행한다.

주요 함수:

- `validate_traceability()`: KSA, evidence claim, parent template, item의 참조 ID가 존재하는지 확인하고, 'item → template → claim → KSA → construct' 관계가 서로 일치하는지 검사한다.
- `validate_scoring_readiness()`: 응답척도와 scoring 메타데이터가 연결되는지 확인한다.
- `validate_redundancy()`: 공백을 제거한 동일 stem이 중복되는지 확인한다.
- `validate_sensitivity()`: 감시 대상 민감 표현이 있는지 확인한다.
- `validate_project()`: 위 게이트를 한 번에 실행한다.

출력 형태:

```
{
  "ok": true,
  "gates": [
    {"code": "traceability", "ok": true},
    {"code": "scoring_readiness", "ok": true},
    {"code": "redundancy", "ok": true},
    {"code": "sensitivity", "ok": true}
  ]
}
```

현재 한계: ID 관계의 구조적 일관성은 검사하지만, 문항 문장의 의미가 실제 construct와 적절하게 정렬되는지는 자동으로 확정하지 않는다. 의미 판단은 전문가 검토 영역이다.

`item\_quality.py`

역할: 문항 문구에서 규칙 기반 위험을 찾는다.

검사 항목:

- 너무 짧거나 긴 stem
- 질문형 문장
- 이중 질문 가능성이 있는 접속어
- 모호한 빈도 또는 강도 표현
- 다중 부정
- 민감 표현
- strict 모드에서 단일 업무 상황과 단일 반응 변수 누락
- 심표로 연결된 복수 상황

주요 함수:

- `audit\_item()`: 문항 하나를 검사한다.
- `audit\_project()`: 모든 문항을 검사한다.
- `audit\_markdown()`: 검사 결과를 Markdown 표로 만든다.

출력에는 `screening\_type = "automated\_rule\_based"`, `requires\_expert\_review = true`, `expert\_review\_criteria`가 포함된다. 즉, 자동 규칙의 결과와 전문가 판단 영역을 데이터 구조에서 구분한다.

현재 한계: 문자열 규칙 기반이므로 의미를 이해하는 검사가 아니다. 오탐과 미탐이 가능하며 전문가 검토를 대체하지 않는다.

`pre\_response.py`

역할: 응답 전 readiness 통합 보고서를 만든다.

이 파일이 현재 설계의 중심이다.

주요 상수:

- `VALIDITY\_BOUNDARY`: 사전 점검이 경험적 타당도를 확정하지 않는다는 경계 문구다.
- `SUPPORTED\_CLAIMS`: 응답 전 단계에서 말할 수 있는 주장이다.
- `REQUIRES\_RESPONSE\_DATA`: 응답자료 이후에만 말할 수 있는 주장이다.

주요 함수:

- `pre\_response\_readiness()`: 구조 점검, 문항 품질 점검, 전문가 검토 상태, blueprint, ECD fit, CAF를 합친다.
- `pre\_response\_markdown()`: readiness 보고서를 읽기 쉬운 Markdown으로 만든다.

상태값:

```
revision_required
ready_for_expert_review
ready_for_pilot_administration
```

`ready\_for\_pilot\_administration`도 경험적 타당도 확보를 의미하지 않는다.

`review.py`

역할: 전문가 검토가 끝났는지 확인한다.

허용 결정값:

```
approve
reject
revise
```

주요 함수:

- ``review_status()``: 문항별 전문가 결정이 있는지 확인한다.
- ``review_markdown()``: 검토 상태를 Markdown으로 만든다.

추가 규칙: 구조 게이트가 실패한 문항을 전문가가 ``approve``한 경우 문제로 기록한다.

``blueprint.py``

역할: 문항 배분과 템플릿 변수 커버리지를 요약한다.

주요 함수:

- ``blueprint_report()``: KSA별 문항 수, 템플릿별 문항 수, 변수별 사용 여부를 계산한다.
- ``blueprint_markdown()``: 결과를 Markdown 표로 만든다.

이 파일은 특정 KSA에 문항이 몰리거나, 템플릿 변수 중 한 번도 사용되지 않은 값이 있는지 살펴볼 때 유용하다.

``ecd_report.py``

역할: 문항이 최소 ECD 계보를 가지고 있는지 간단히 요약한다.

주요 함수:

- ``ecd_fit_report()``: 각 문항에 ``construct_id``, ``ksa_id``, ``evidence_claim_id``, ``parent_template_id``가 있는지 센다.

주의점: 이 보고서는 빠른 요약이다. 더 엄격한 구조 검사는 ``validation.validate_traceability()``가 담당한다.

``caf.py``

역할: CAF 관점에서 설계 지도를 만든다.

출력 구성:

- ``student_model``
- ``evidence_model``
- ``task_model``
- ``task_evidence_composite``
- ``four_process_architecture``
- ``validity_boundary``

``four_process_architecture``는 활동 선택, 과제 제시, 응답 처리, 요약 채점의 흐름을 설명한다.

``toulmin.py``

역할: 특정 문항과 응답을 Toulmin 논증 구조로 설명한다.

출력 구성:

- ``claim``: 무엇을 주장하는가
- ``grounds``: 문항, 응답, 점수, 변수
- ``warrant``: 왜 이 응답을 근거로 해석하는가
- ``backing``: 어떤 설계 계보가 뒷받침하는가

- `qualifier`: 해석 범위
- `rebuttals`: 반론과 한계

이 파일의 핵심은 자동 생성 문항을 무조건 신뢰하는 것이 아니라, 해석의 전제와 반론을 명시하는 것이다.

`dossier.py`

역할: 여러 보고서를 하나의 문항 근거 묶음으로 합친다.

포함 항목:

- 구조 검증
- 문항 품질 검사
- blueprint
- 전문가 검토 상태
- ECD fit
- CAF
- Toulmin 논증
- validity boundary

주요 함수:

- `dossier()`: 통합 딕셔너리를 만든다.
- `dossier\_markdown()`: 간단한 읽기용 보고서를 만든다.

`scoring.py`

역할: 응답을 점수로 바꾼다.

주요 함수:

- `parse\_response()`: 'A'부터 'E' 숫자, 척도 라벨을 점수로 해석한다.
- `score\_response()`: 문항 scoring 방향을 적용한다.
- `response\_scale\_parity()`: 척도 점수가 빠짐없이 연속적인지 확인한다.

역채점:

```
1, 2, 3, 4, 5 척도에서 raw=2이고 direction="reverse"
→ score = 5 + 1 - 2 = 4
```

`response\_data.py`

역할: 파일럿 응답 CSV를 읽고 점수화 가능한지 확인한다.

주요 함수:

- `load\_responses()`: UTF-8 BOM을 허용하는 CSV 로더다.
- `validate\_responses()`: 문항별 응답을 점수로 변환하고 오류를 수집한다.

빈 응답은 `None`으로 유지한다. 이 파일부터는 응답자료 이후 단계다.

`psychometrics.py`

역할: 파일럿 응답자료에서 기초 CTT 통계를 계산한다.

주요 함수:

- `variance()`: 표본분산을 계산한다.

- ``corr()``: 피어슨 상관을 계산한다.
- ``psychometrics_report()``: 문항 평균, 표준편차, 결측 수, 수정 문항-총점 상관, Cronbach alpha를 계산한다.
- ``psychometrics_markdown()``: 결과를 Markdown 표로 만든다.

결측 처리:

- 수정 문항-총점 상관은 해당 문항에 응답한 동일 응답자의 문항값과 rest total만 짝지어 계산한다.
- Cronbach alpha는 모든 문항에 응답한 complete case만 사용한다.
- 결과의 ``alpha_respondents``는 alpha 계산에 포함된 응답자 수를 보여 준다.
- 부동소수점 반올림 때문에 상관계수가 아주 미세하게 범위를 벗어나지 않도록 ``-1.0``에서 ``1.0`` 사이로 제한한다.
- 전체 결측률이 5%를 넘으면 경고를 낸다.
- alpha complete-case 비율이 80%보다 낮으면 경고를 낸다.
- alpha를 계산할 수 없으면 별도 경고를 낸다.

현재 한계:

- 완전한 심리측정 패키지가 아니다.
- IRT와 DIF 분석은 포함하지 않는다.

``agreement.py``

역할: 사람이 정한 gold label과 agent label의 일치도를 계산한다.

주요 함수:

- ``agreement_report()``: 정확도, Cohen kappa, confusion count를 반환한다.

이 기능은 문항 품질 판정 규칙 또는 자동 판정 결과를 사람이 정한 기준과 비교할 때 사용할 수 있다.

``simulation.py``

역할: 부모 템플릿에서 만들 수 있는 변수 조합 수를 계산한다.

주요 함수:

- ``simulation_summary()``: 템플릿별 가능한 조합 수, 현재 문항 수, 커버리지 비율을 반환한다.

이 파일은 실제 문항을 생성하지 않고 생성 공간의 크기를 요약한다.

``export.py``

역할: 후보 문항을 외부 파일로 내보낸다.

지원 형식:

```
csv
json
lms-json
qti-lite
```

주요 함수:

- ``item_records()``: 내보내기 공통 레코드 형태를 만든다.
- ``export_items()``: 형식에 따라 CSV, JSON, XML을 작성한다.

주의점: ``qti-lite``는 완전한 QTI 구현이 아니라 경량 프로토타입이다.

``qti-lite`` 출력 보고서에는 ``profile = "prototype_qti_lite"``와 ``production_lms_ready = false``가 포함된다. 실제 LMS 운영 연동 전에는 정식 IMS QTI 패키지 요구사항을 별도로 검토해야 한다.

`rendering.py`

역할: CLI 보고서 출력 형식을 보조한다.

주요 함수:

- `emit\_json()`: UTF-8 문자를 유지하는 JSON 문자열을 만든다.
- `table()`: Markdown 표 문자열을 만든다.
- `status\_mark()`: 불리언 값을 `PASS` 또는 `FAIL`로 바꾼다.

`webapp.py`

역할: 별도 웹 프레임워크 없이 로컬 확인 화면을 제공한다.

구성:

- `ThreadingHTTPServer`: 로컬 HTTP 서버
- `available\_projects()`: `examples` 안에서 `items` 목록을 가진 JSON만 프로젝트 목록으로 만든다.
- `resolve\_project()`: `examples` 밖의 파일을 요청하지 못하도록 경로를 제한한다.
- `Handler.do\_GET()`: `/api/projects`, `/api/project` JSON API를 제공하고 오류는 400 JSON으로 반환한다.
- `HTML`: 프로젝트 선택, readiness 요약, 문항 목록, 오류 화면을 제공한다.
- `run()`: 기본 주소 `127.0.0.1:8765`에서 서버 실행

화면은 사용자 데이터를 HTML 문자열로 삽입하지 않고 DOM `textContent`로 렌더링한다.

주의점: 연구용 로컬 프로토타입이다. 인증과 권한관리를 갖춘 운영용 웹 애플리케이션은 아니다.

7. 모듈 관계 요약

```
models
├── generation
├── import_items
├── validation
├── item_quality
├── scoring
├── response_data
└── 대부분의 보고서 모듈
```

```
pre_response
├── validation
├── item_quality
├── review
├── blueprint
├── ecd_report
└── caf
```

```
dossier
├── validation
├── item_quality
├── blueprint
├── review
├── ecd_report
├── caf
└── toulmin
```

```
psychometrics
→ response_data
→ scoring
```

```
webapp
→ models.load_project
```

→ `pre_response.pre_response_readiness`

`export`

→ `models.Project`

→ CSV / JSON / prototype QTI-lite

`tests`

→ 정상 프로젝트 회귀 테스트

→ 구조 게이트별 실패 fixture

→ 결측 응답 경고 fixture

→ QTI-lite 범위 표시

→ 웹 경로 제한

8. 반영된 보완 사항과 다음 단계

이번 버전에는 다음 보완 사항이 반영되어 있다.

1. ``validation.py``: 계보 ID 존재 여부를 넘어 관계 일관성 검사를 추가했다.
2. ``item_quality.py``: 규칙 기반 문자열 검사와 전문가 판단의 역할을 출력 구조에서 분리했다.
3. ``psychometrics.py``: 결측 처리 정책, complete-case 기준, 경고 임계값을 추가했다.
4. ``webapp.py``: 안전한 프로젝트 선택, DOM ``textContent`` 렌더링, 오류 화면을 추가했다.
5. ``export.py``: QTI-lite가 운영용 정식 QTI 패키지가 아님을 출력 메타데이터에 명시했다.
6. 테스트: 구조 게이트별 실패 fixture와 결측 경고 테스트를 추가했다.

다음 단계에서 검토할 사항:

1. 의미 기반 construct alignment 검토를 위한 전문가 체크리스트 저장 구조
2. 실제 LMS 대상 정식 IMS QTI 패키징
3. 운영 웹 서비스가 필요할 경우 인증, 권한관리, 프로젝트 업로드 기능
4. 파일럿 자료 규모가 커질 경우 결측 처리 옵션과 분석 보고서 확장

9. 가장 중요한 결론

이 코드는 자동 생성 문항을 곧바로 신뢰하기 위한 코드가 아니다.

AI가 문항을 만들었다

→ 그러므로 타당하다

가 아니라 다음 구조를 구현한다.

AI가 후보 문항을 만들었다

→ 설계 계보를 기록한다

→ 구조적 위험을 탐지한다

→ 전문가가 검토한다

→ 파일럿 응답자료를 수집한다

→ 이후에 실증 분석을 수행한다

따라서 현재 코드의 정확한 역할은 자동문항생성기, 설계 추적기, 응답 전 위험 점검기, 실증 검증 준비 도구의 결합이다.